

Lecturer: Yetmwork Tesfaye

MSc. in Computer Science and Engineering

College of Computing and Informatics

Department of Computer Science

Chapter 3: Process

Objectives

At the end of this Chapter Students be able to know:-

- ✓ Introduction to Threads
- ✓ Threads in Distributed Systems
- ✓ Clients
- ✓ Servers
- ✓ Code Migration

Introduction to Threads

What is Process?

- A program in execution.
- A program that is currently being executed on one of the operating system's virtual processors.
- A process includes a program code and its state(new (Created), Ready, Running, Waiting, Exit(Terminated)).
- A single CPU can Run multiple processes at the same time(Multitasking)
- A single Process may Contain one or More Threads.

Introduction to Threads

What is Thread?

- An independent Sequence of Instructions.
- Smallest sequence of programmed instructions that can be managed independently by a scheduler(OS Component).
- A thread is **created** by a **Process**.
- A thread is a **Subset** of the **Process**.
- Multiple threads share the **process instructions**, **process variables** and **process states**.

Introduction to Threads

Advantages of Threads

➤ Parallelism

- E.g. **Spreadsheets**, while one thread is working for input, the other threads can do other computations.
- Client Applications. E.g. **Web browsers**
 - While one thread works for **communicating data**, the others can work on displaying the **received data**.

Threads in Distributed Systems



- Threads in distributed system are implemented in the form **multithreading**.
- A single CPU can run multiple **threads** concurrently.
- A CPU with multiple Cores or multiple processors can actually run multiple threads in parallel on different cores or processors.
- **Advantages of Multithreading:** Improved Performance, Concurrency, Resource Utilization, Asynchronous Operations, Speed...
- **Disadvanges of Multithreading:** Mutual Exclusive(must be implemented)

Threads in Distributed Systems

(1.Multithreaded Clients)

- Main issue is hiding network latency.
- Multithreaded web client:
 - Web document consists of an HTML file containing plain text along with a collection of images, icons etc.
 - To fetch each element of a Web document, the browser has to set up a TCP/IP connection
 - Browser scans HTML, and finds more files that need to be fetched.
 - Each file is fetched by a separate thread, each issuing an HTTP request.
 - As files come in, the browser displays them.

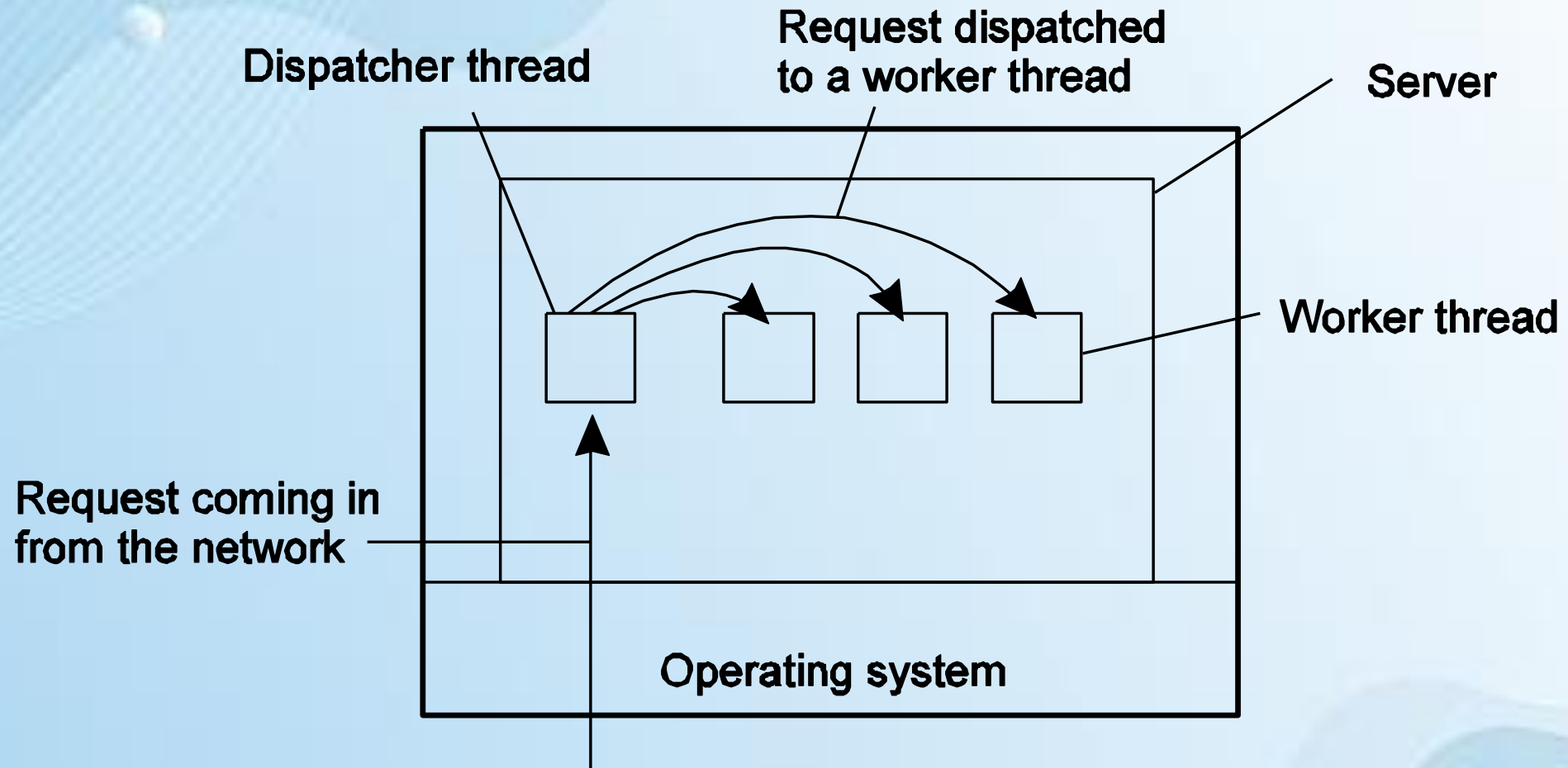
Threads in Distributed Systems

(2.Multithreaded Servers)

- The main use of multithreading in distributed systems is found at the server side.
- Multithreading allows to develop servers that exploit parallelism to attain high performance.
- Parallelism improve performance

Threads in Distributed Systems

(2.Multithreaded Servers(e,g file server)



- A multithreaded server organized in a dispatcher/worker model.

Clients(what is Client?)

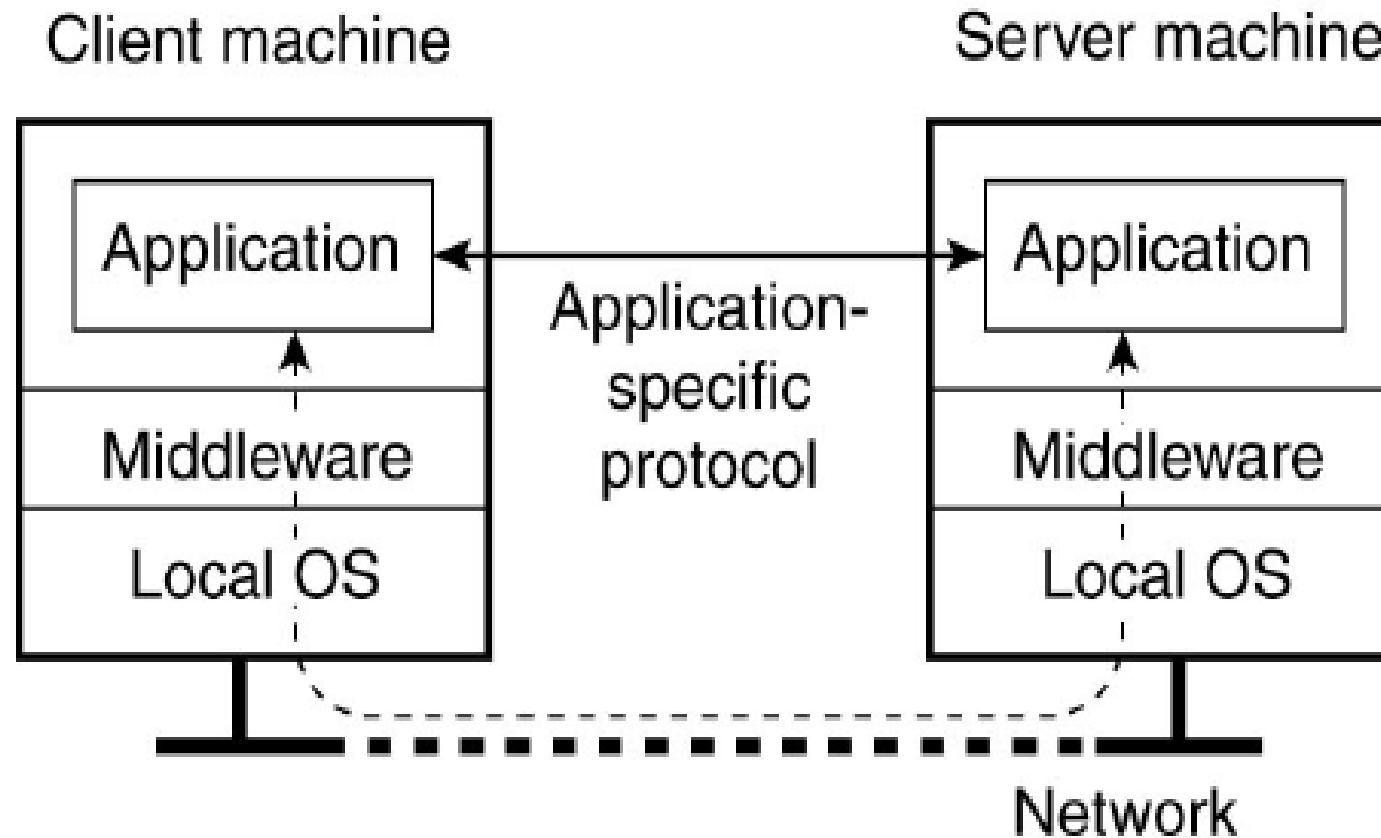


- A program which **interacts** with a human user and a remote server.
- Typically, the user interacts with the client via a GUI.
- A major task of client machines is to provide the means for users to interact with remote servers.

Clients(what is Client?)

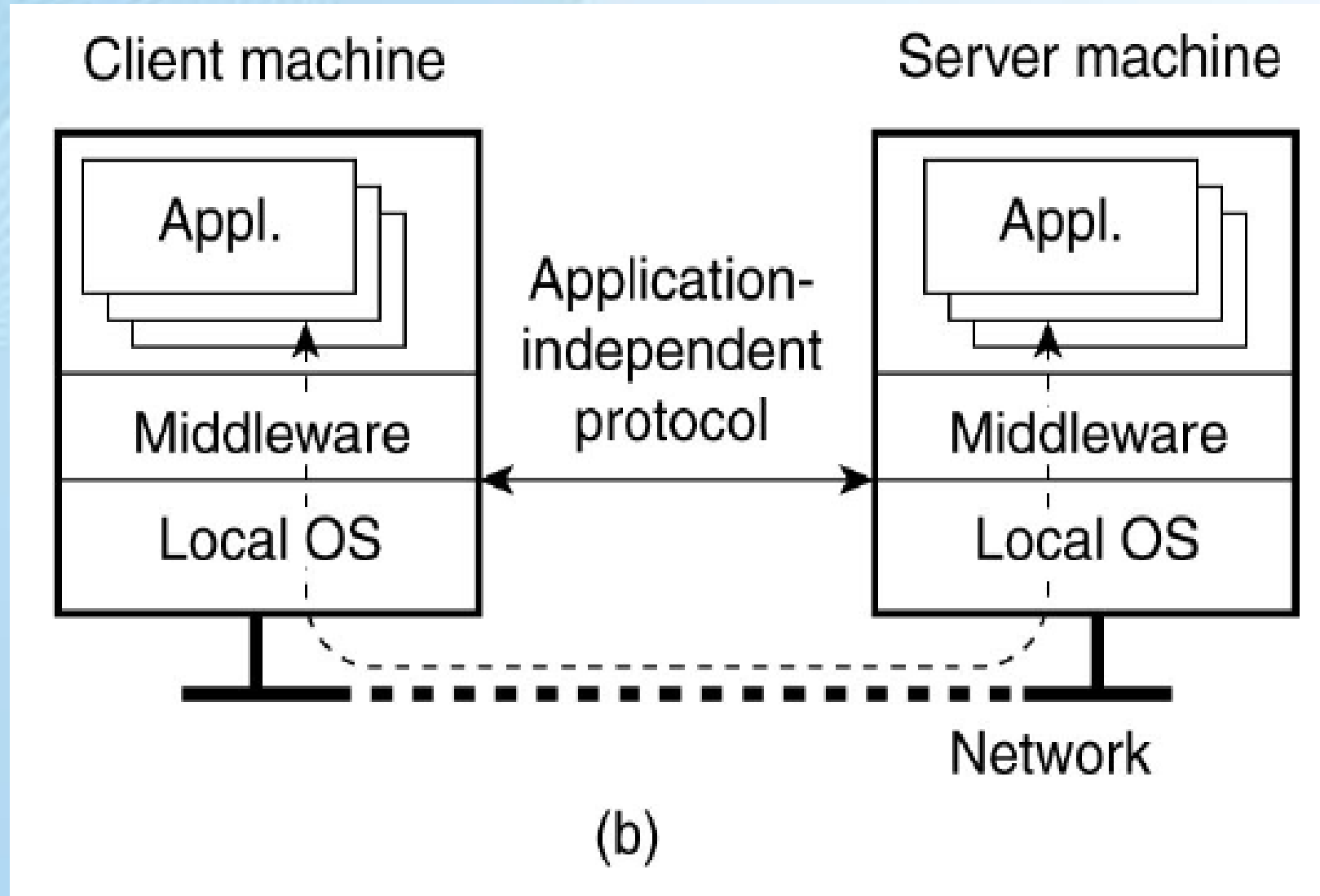
- There are roughly two ways in which this interaction can be supported.
 1. For every application, create a client part and a server part.
 - Client runs on local machine, such as a PDA.
 2. Create a reusable GUI toolkit that runs on the client. GUI can be directly manipulated by the server-side application code.
 - E.g :-X Window System

Clients(Networked Interfaces(1))



A networked application with its own protocol

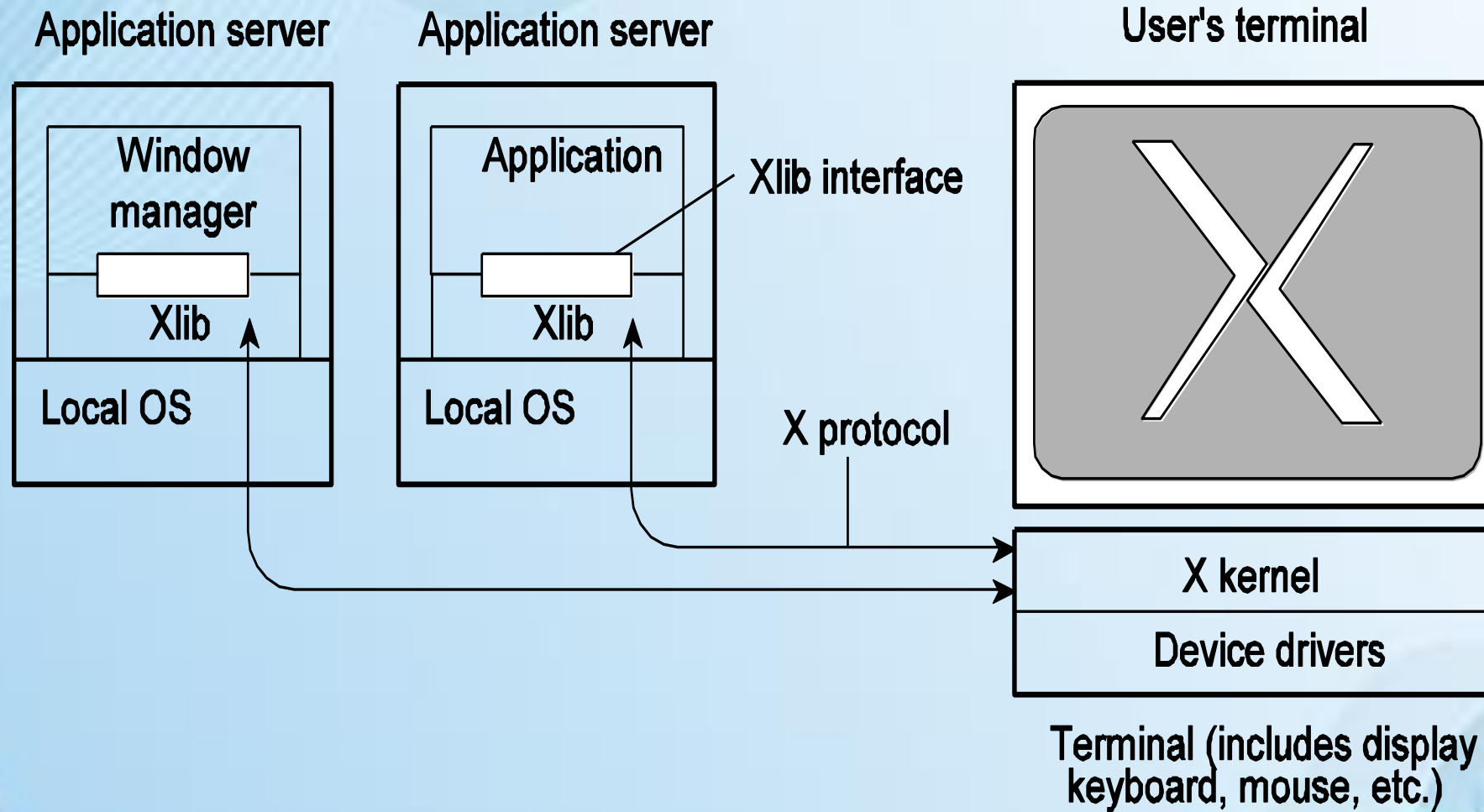
Clients(Networked Interfaces(2))



X Window System

- The X Window System is a network-transparent graphical windowing system based on a client/server model.
- Primarily used on Unix and Unix-like systems such as Linux
- versions of X are also available for many other operating systems.
 - Although it was developed in 1984, X is not only still available but also is in fact the standard environment for Unix windowing systems.

The X Window System

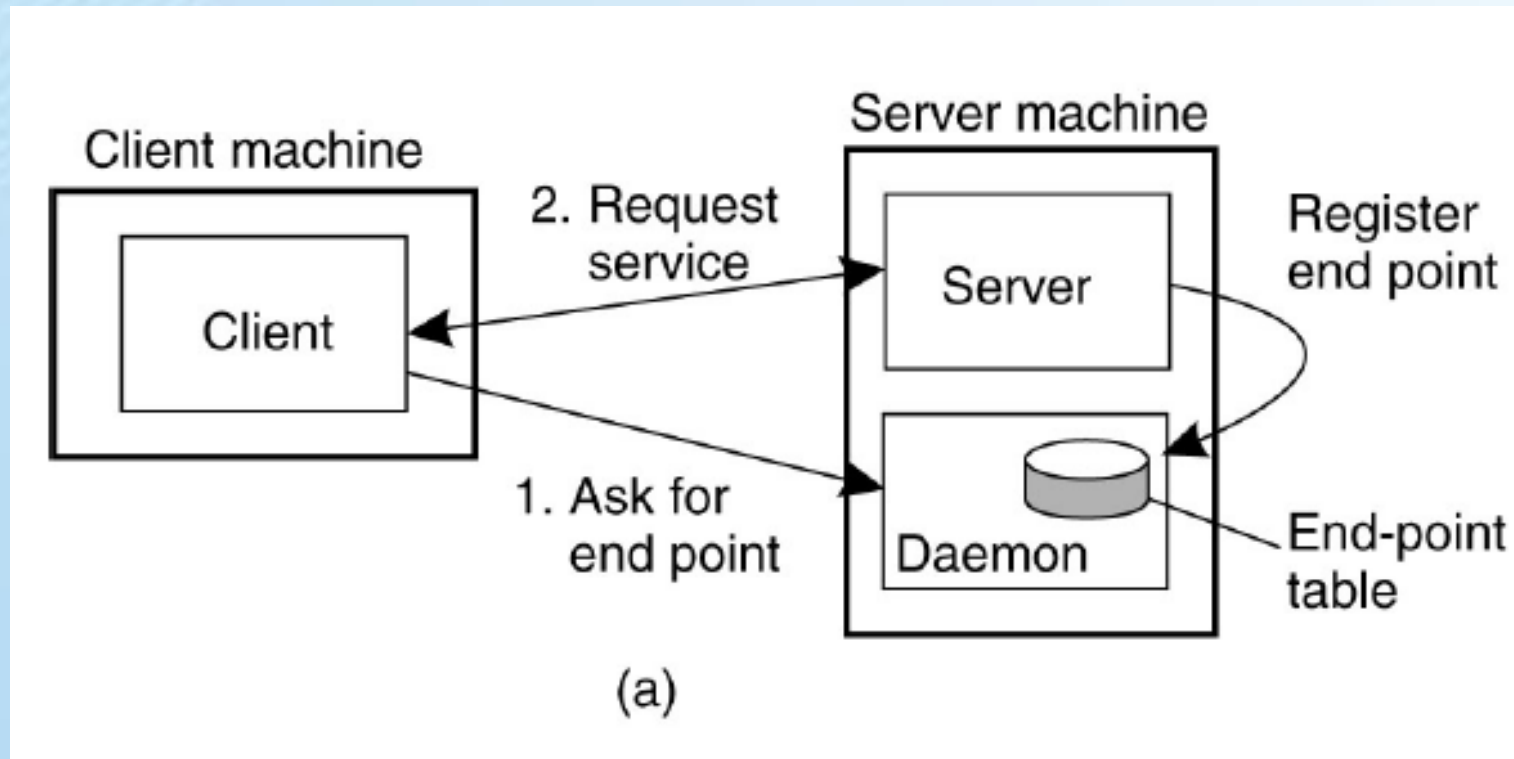


Servers: What's a Server?

- A process that implements a specific service **on behalf of** a collection of clients.
- Typically, servers are organized to do one of two things:
 1. Wait
 2. Service... wait ... service ... wait ... service ... wait ...

Servers(General Design Issues)

Passive: servers wait until a request arrives



Servers(General Design Issues)

There are 4 main Design issues for Servers

1. Server Design
2. How do clients know the end point of a service ?
3. Server Interrupt
4. Server State

SERVERS(1.Server Design)

- **Iterative:** server handles request, then returns results to the client; any new client requests must wait for previous request to complete.
- **Concurrent:** server does not handle the request itself; a separate thread or sub-process handles the request and returns any results to the client; the server is then free to immediately service the next client
- **Super servers:** Listen to multiple end points, then spawn the right server.

SERVERS(2.Client contact Servers)

- In all cases, clients send requests to an end point, also called a port, at the machine where the server is running.
- Each server listens to a specific end point.
- Problem: Identifying “end-points”?
- **How do clients know the end point of a service?**

Two options:

- ❖ **Statically assigned end-points** (IANA, Internet Assigned Numbers Authority).
 - E.g:- servers that handle Internet FTP requests always listen to TCP port 21. Likewise, an HTTP server for the World Wide Web will always listen to TCP port 80

SERVERS(2.Client Contact Servers)

Challenges:

1. There are many services that do not require a preassigned end point.
 - Soln: use DCE or Daemon(see fig 3.11(a))
2. Implementing each service by means of a separate server may be a waste of resources.
 - E.g:- lots of servers running simultaneously, with most of them passively waiting until a client request comes in.
 - Soln:-use Super Server(see fig 3.11(b))

SERVERS(2.client contact Servers)

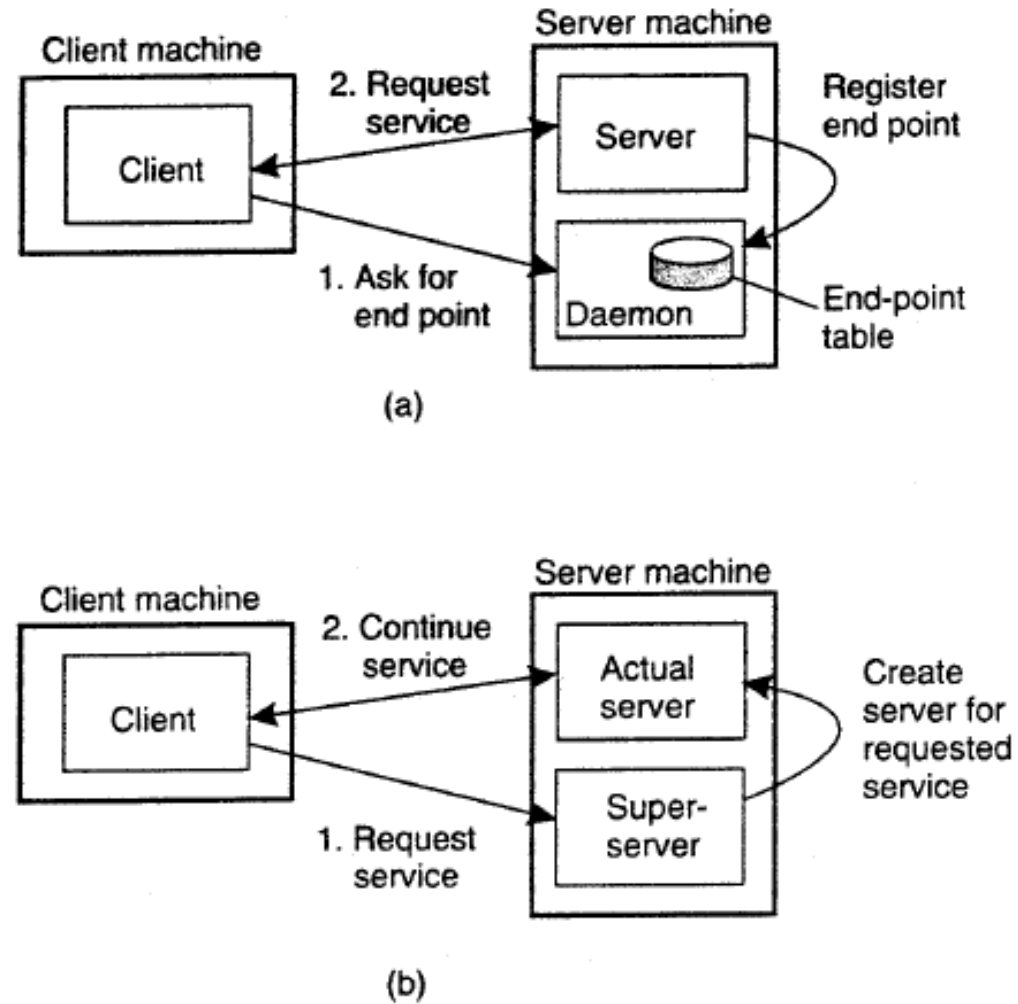


Figure 3-11. (a) Client-to-server binding using a daemon. (b) Client-to-server binding using a superserver..

Server Design

(3.Server interruption)

- Sometimes there is a situation in which server interruption is required.
E.g:- consider a user who has just decided to upload a huge file to an FTP server.
- Then, suddenly realizing that it is the wrong file, he wants to interrupt the server to cancel further data transmission.

How to Interrupt servers?

Server Design

(3.Server interruption)

There are several ways to do this.

1. The user can abruptly exit the client application immediately restart it, and pretend nothing happened.

- The server will eventually tear down the old connection, thinking the client has probably crashed.

2. Out-of-band data communication

- Send **out-of-band data which** is data that is to be processed by the server before any other data from that client.

Server Design(Server States)

➤ Stateless servers

- no information is maintained on the current “connections” to the server.
- do not store any client-specific data or session state between requests.
 - Example: web server.

➤ Stateful servers

- information is maintained on the current “connections” to the server.
- store client-specific data and session state between requests.
 - Example: file server.

Server Clusters

What is Server Clusters?

- A collection of machines connected through a network, where each machine runs one or more servers.
- a server cluster is logically organized into three tiers.
- **First tier** consists of a switch through which client requests are routed.
- **Second Tier** contains servers dedicated to application processing.
- **Third Tier** consists of data-processing servers, notably file and database servers.

Server Clusters

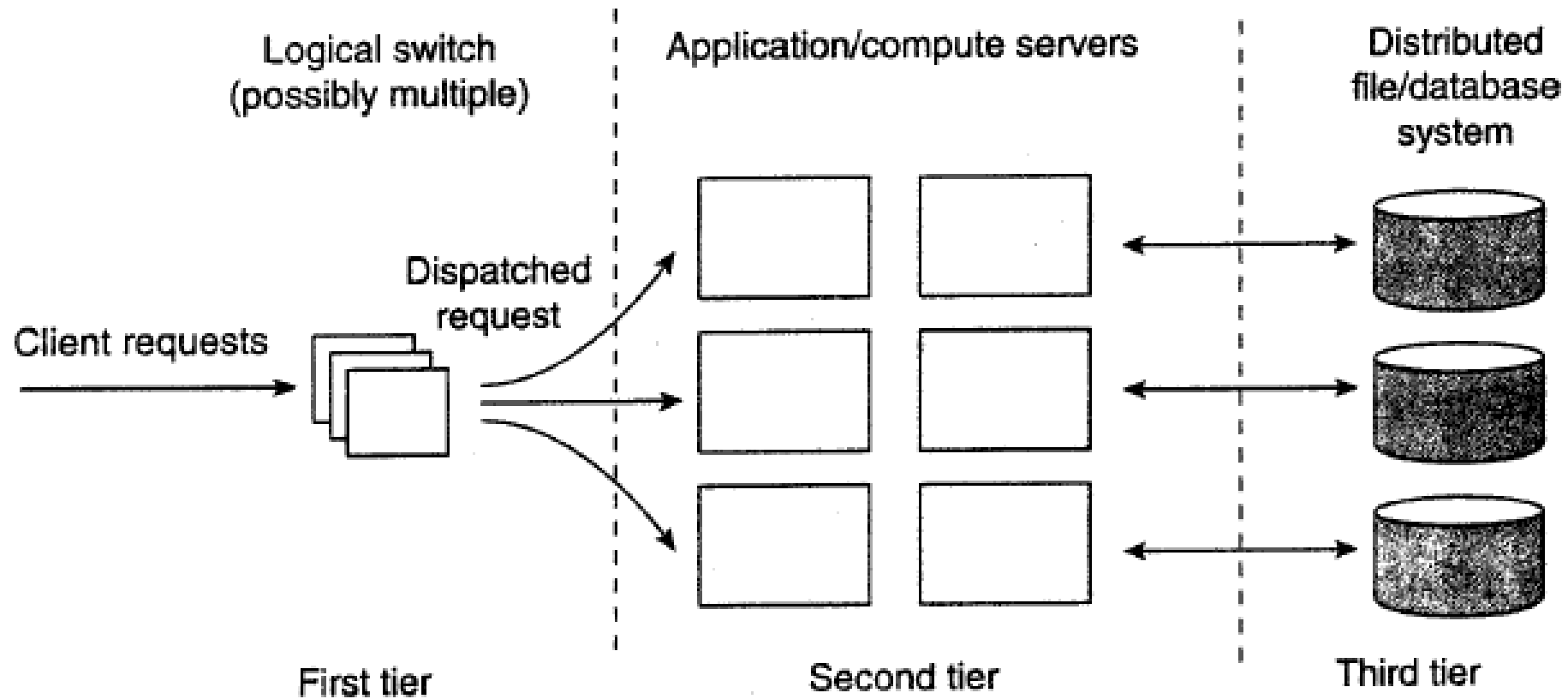


Figure 3-12. The general organization of a three-tiered server cluster.

Server Clusters

- A standard way of accessing a server cluster is to set up a TCP connection over which application-level requests are then sent as part of a session.

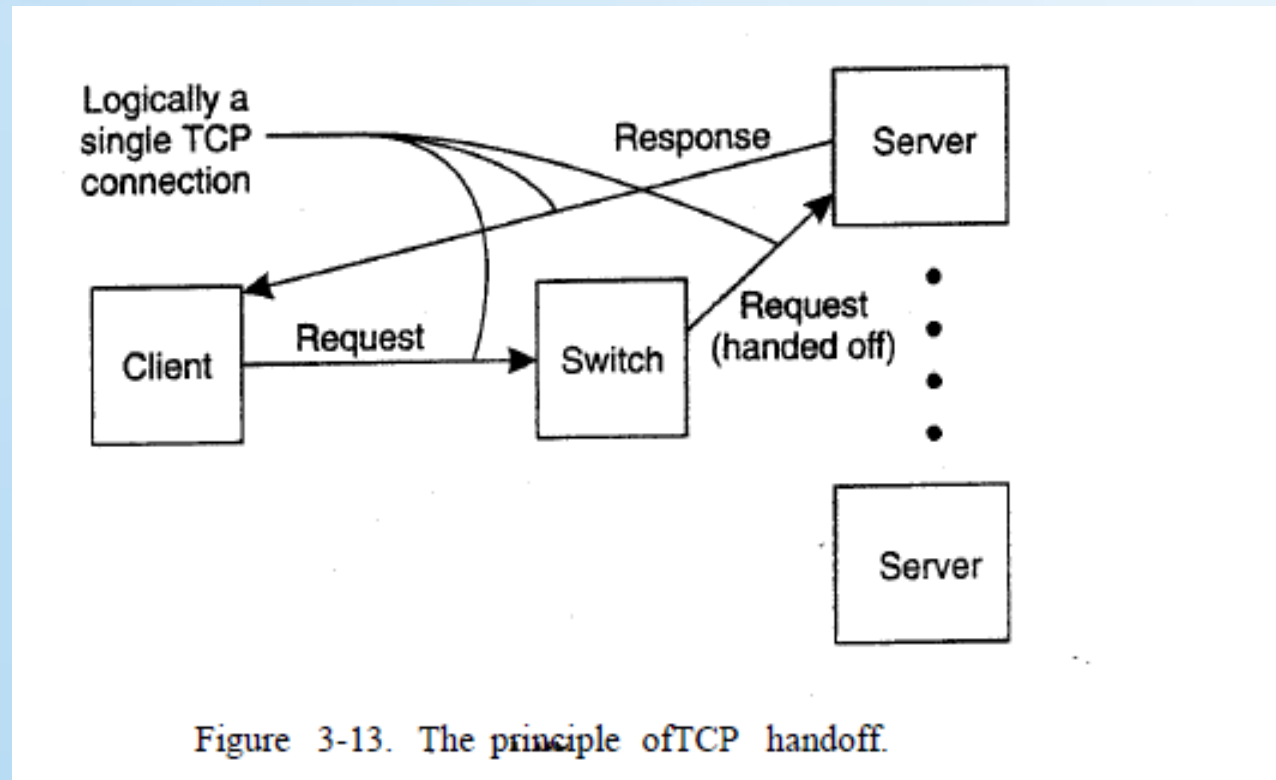


Figure 3-13. The principle of TCP handoff.

Code Migration

What is Code Migration?

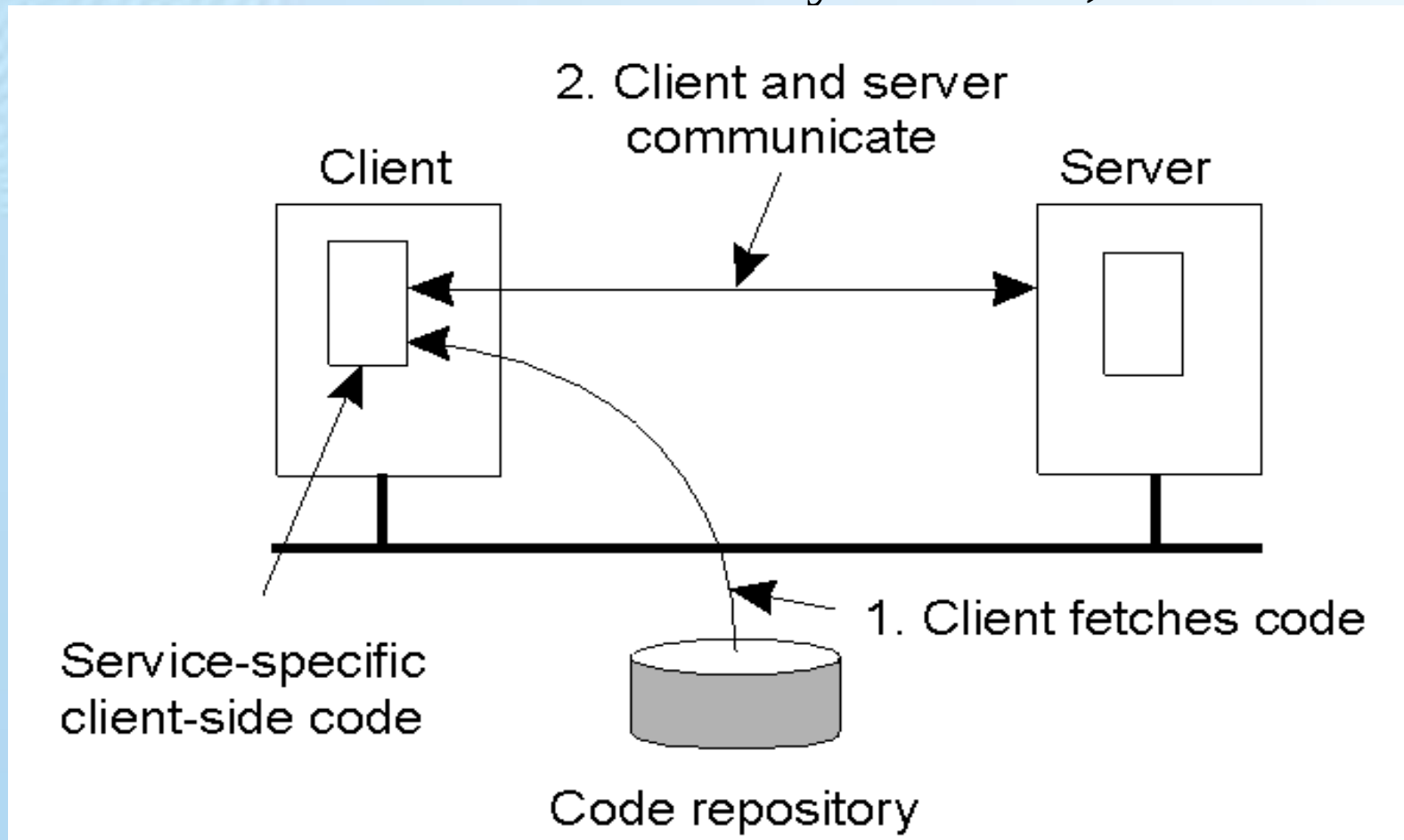
- Under certain circumstances, in addition to the usual passing of data, **passing code** (even while it is executing) can greatly simplify the design of a distributed system.
- Code migration is when programs are moved from one machine to another, often moving parts of its execution environment along with it.
- The intent is for the transferred program to continue execution on the target machine.

Reasons For Migrating Codes

- Biggest single reason: **better performance**.
- The big idea is to move a compute-intensive task from a heavily loaded machine to a lightly loaded machine “on demand” and “as required”.
- Flexibility
 - Allow to dynamically configure a distributed system

Reasons For Migrating Codes (Dynamically Configuring a Client)

- The client first fetches the necessary software, and then invokes the server



Code Migration Model

- A running process consists of three “segments”:
 - **Code Segment** – instructions
 - The part that contains the set of instructions that make up the program that is being executed.
 - **Resource Segment** – external references.
 - The part that contains references to external resources needed by the process, such as files, printers, devices, other processes, and so on.
 - **Execution Segment** – current state.
 - Used to store the current execution state of a process, consisting of private data, the stack, and the program counter.

Code Migration Characteristics

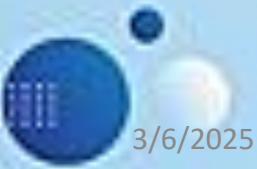


- **Weak Mobility:** transfer only the **code segment** – and a program always starts from its initial stage
- **Strong Mobility:** the **code** and **execution segment** is moved – and execution restarts from the next statement.
 - strong mobility also supports the notion of “remote cloning”: an exact copy of the original process, but now running on a different machine.

More Characteristics(Code Migration)



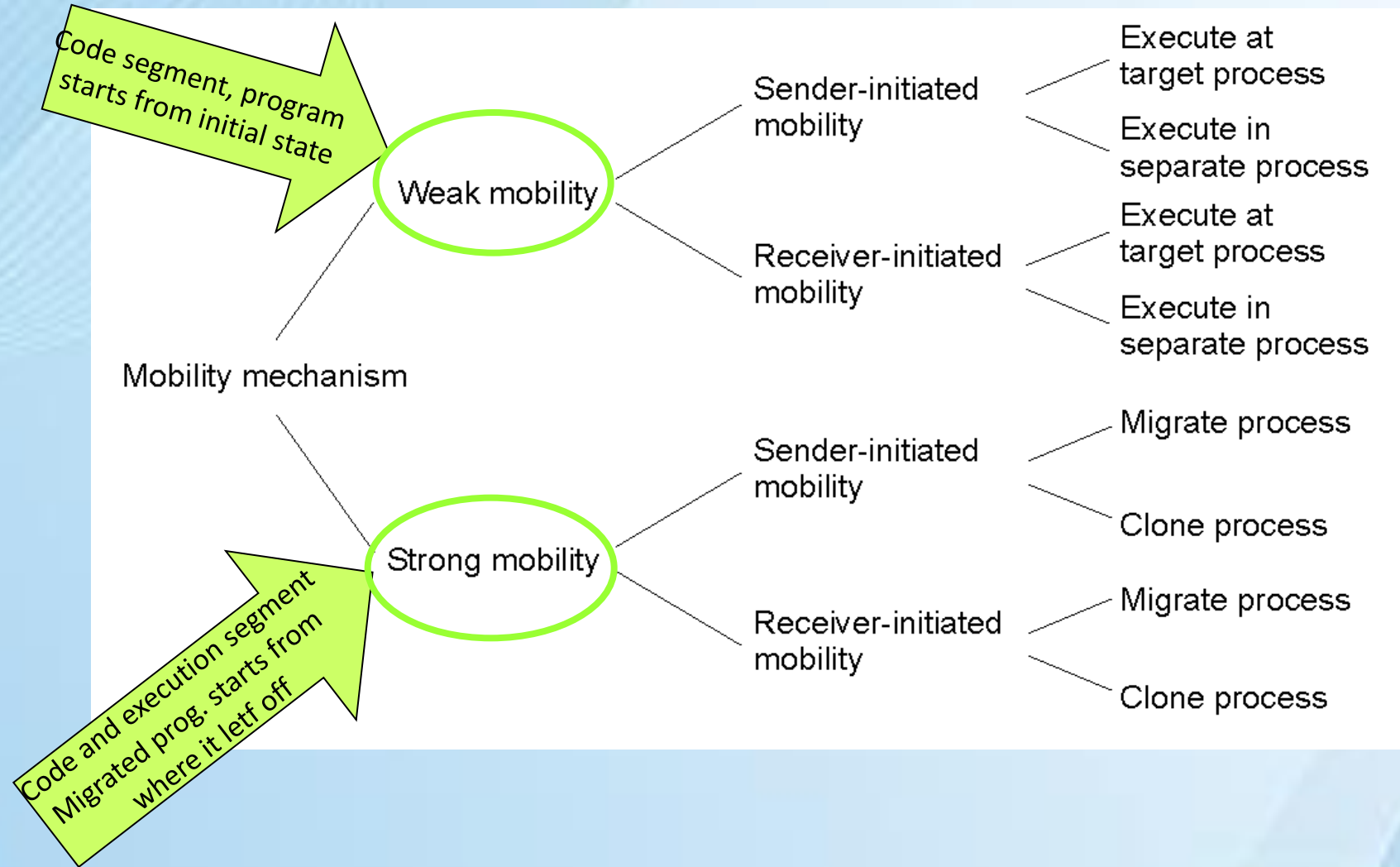
- **Where migration starts ?**
- Sender vs Receiver-Initiated.
 - Which side of the communication starts the migration?
- **Sender-initiated:** migration initiate at the machine where the code resides.
 - E.g Uploading programs to a server; may need authentication or that the client is a registered one
- **Receiver initiated:** migration starts by target machine initiative
 - E.g: Java applets



How Does the Migrated Code Run?

- Another issue surrounds where the migrated code executes:
 - Within an existing process (possibly as a thread)
 - or
 - Within it's own (new) process space.

Models for Code Migration



What About Resources?

Do Resources migrate?

- What makes code migration difficult is the requirement to migrate resources.
- Resources are the external references that a process is currently using.

Examples: Variables, open files, network connections, printers, databases, etc.

Resource Migration: Types of Process-to-Resource Binding

- **Strongest:** Binding-by-identifier (BI) – Process refers to resource by its identifier
 - precisely the referenced resource, and nothing else, has to be migrated.
- **Binding-by-value (BV)** –only the value of the resource need be migrated.
- **Binding-by-type (BT)** – nothing is migrated, but a resource of a specific type needs to be available after migration (e.g, a printer).

Resource Migration: Resource-to-machine Binding

Resources are further distinguished as one of:

1. **Unattached**: a resource that can be moved easily from machine to machine.
2. **Fastened**: migration is possible, but at a high cost.
3. **Fixed**: a resource is bound to a specific machine or environment, and cannot be migrated.

Thank You!!